Universidade do Algarve

Faculdade de Ciências e Tecnologia

Licenciatura em Engenharia Informática

ALGORITMOS E ESTRUTURAS DE DADOS

Parte C – Análise e relatório

Projeto 3 – Tabelas de Símbolos

Implementação de uma UAlgTree e UAlshTable

Realizado por: Diogo Freitas nº 90147, Diogo Carvalho nº 90247.

Turma/Grupo: PL1-G10.

Regente da Disciplina: João Miguel Dias

Responsável pelas Aulas Práticas: Nestor Cataño Collazos

Ano Letivo: 2025/2026

1) Parte A - UAlgTree

1.1) Introdução

Nesta secção iremos analisar o funcionamento do UAlgTree, implementada no Problema A, com o objetivo de avaliar o seu balanceamento e a eficiência das operações. O estudo foca-se em dois aspetos principais: (1) a robustez da árvore face a diferentes ordens de inserção, através da análise da profundidade mínima e máxima após inserir um número elevado de chaves, e (2) o impacto da estratégia de promoção de chaves durante get, comparando pesquisas com distribuição uniforme e com um padrão 80/20 inspirado na regra de Pareto. Os resultados obtidos permitem caracterizar o comportamento da estrutura em cenários típicos e adversos, e relacioná-lo com a eficiência esperada teoricamente.

1.2) Metodologia

1.2.1) Balanceamento

Implementamos dois métodos auxiliares para medir a profundidade mínima (minDepth) e máxima (maxDepth) a partir da raiz até uma folha.

O balanceamento foi calculado pelo rácio: $\frac{\text{maxDepth}}{\text{minDepth}}$.

Foram realizados testes com $N = 100000$ inserções em três cenários:

- Chaves aleatórias (5 execuções com sementes distintas);
- Chaves inseridas por ordem crescente;
- Chaves inseridas por ordem decrescente.

1.2.2) Pesquisas

O desempenho de pesquisa foi analisado através de testes de **razão dobrada** para $N = 50\,000, 100\,000, 200\,000$ e $400\,000$.

Em cada tamanho foram efetuadas **300 000** operações get, medindo-se o tempo médio por pesquisa em nanossegundos.

Utilizamos 2 tipos de testes:

- uniforme, onde todas as chaves têm a mesma probabilidade de pesquisa;
- 80/20, onde 20% das chaves são pesquisadas em 80% das vezes.

1.3) Resultados

1.3.1) Balanceamento

O Testes com para $N = 100000$

```
Random insert (single run)
N = 100000
seed = 12345
minDepth = 12
maxDepth = 22
ratio    = 1.8333333333333333
```

```
Sorted increasing insert
N = 100000
minDepth = 16
maxDepth = 21
ratio    = 1.3125
```

```
Sorted decreasing insert
N = 100000
minDepth = 16
maxDepth = 21
ratio    = 1.3125
```

```
Random insert (multiple runs)
N = 100000, runs = 5

seed = 12345
minDepth = 12
maxDepth = 22
ratio    = 1.8333333333333333

seed = 12346
minDepth = 13
maxDepth = 21
ratio    = 1.6153846153846154

seed = 12347
minDepth = 13
maxDepth = 22
ratio    = 1.6923076923076923

seed = 12348
minDepth = 13
maxDepth = 22
ratio    = 1.6923076923076923

seed = 12349
minDepth = 13
maxDepth = 21
ratio    = 1.6153846153846154
```

1.3.2) Pesquisas com distribuição uniforme

```
Search Uniform (Doubling Ratio)
queries per N = 300000

N=50000 | avg=219,9 ns/get
N=100000 | avg=265,0 ns/get | R=1,205
N=200000 | avg=377,3 ns/get | R=1,424
N=400000 | avg=591,3 ns/get | R=1,567
```

1.3.3) Pesquisas com padrão 80/20

```
Search Pareto 80/20 (Doubling Ratio)
queries per N = 300000

N=50000 | avg=214,3 ns/get
N=100000 | avg=244,3 ns/get | R=1,140
N=200000 | avg=310,0 ns/get | R=1,269
N=400000 | avg=468,1 ns/get | R=1,510
```

1.4) Discussão

Os resultados dos testes de balanceamento mostram que a UAlgTree mantém um grau de equilíbrio consistente em inserções aleatórias e não evidencia degradação quando sujeita a inserção ordenada crescente ou decrescente.

O rácio max/min obtido nos cenários ordenados (1.3125) é inferior ao observado no caso aleatório, sugerindo que o mecanismo de balanceamento é robusto face à ordem de inserção.

Nos testes de pesquisa, as razões dobradas mantiveram-se abaixo de 2 tanto no padrão uniforme como no 80/20, o que indica crescimento sublinear do tempo médio de pesquisa com o aumento de N. O padrão 80/20 apresentou desempenho geral semelhante ao uniforme, com ganhos moderados em alguns tamanhos.

2) Parte B - UAlgTable

2.1) Introdução

Nesta parte iremos analisar a estrutura de dispersão **UAlshTable** (Problema B), com foco no desempenho das operações de inserção e pesquisa. Em particular, pretende-se observar o comportamento da inserção ao longo do crescimento da tabela e estimar o crescimento temporal médio das pesquisas através de testes de razão dobrada, comparando padrões de acesso uniforme e 80/20, bem como pesquisas de chaves ausentes.

2.2) Metodologia

Foram realizados três conjuntos de testes:

1. **Inserção por blocos:** foram efetuadas 200 000 inserções em blocos de 20 000, medindo-se o tempo médio por operação put em cada bloco.
2. **Pesquisas com razão dobrada:** para $N = 50\,000, 100\,000, 200\,000$ e $400\,000$, foram realizadas 300 000 pesquisas por tamanho, medindo-se o tempo médio por operação contains.
3. **Pesquisas de ausência:** com os mesmos tamanhos N, foram testadas chaves fora do universo inserido, com 300 000 pesquisas por tamanho.

2.3) Resultados

2.3.1) Inserção por blocos

```
B - Insertion Load (by blocks)
total inserts = 200000, block = 20000

inserted=20000 | avg=1251,4 ns/put | size=20000
inserted=40000 | avg=399,1 ns/put | size=40000
inserted=60000 | avg=149,1 ns/put | size=60000
inserted=80000 | avg=770,9 ns/put | size=80000
inserted=100000 | avg=169,5 ns/put | size=100000
inserted=120000 | avg=144,9 ns/put | size=120000
inserted=140000 | avg=136,7 ns/put | size=140000
inserted=160000 | avg=1497,4 ns/put | size=160000
inserted=180000 | avg=180,3 ns/put | size=180000
inserted=200000 | avg=181,8 ns/put | size=200000
```

2.3.2) Pesquisas com distribuição uniforme (razão dobrada)

```
B - Search Uniform (Doubling)
queries per N = 300000

N=50000 | avg=61,0 ns/contains
N=100000 | avg=69,2 ns/contains | R=1,134
N=200000 | avg=111,1 ns/contains | R=1,605
N=400000 | avg=118,4 ns/contains | R=1,066
```

2.3.3) Pesquisas com padrão 80/20 (razão dobrada)

```
B - Search Pareto 80/20 (Doubling)
queries per N = 300000

N=50000 | avg=85,3 ns/contains
N=100000 | avg=127,5 ns/contains | R=1,495
N=200000 | avg=131,5 ns/contains | R=1,031
N=400000 | avg=230,9 ns/contains | R=1,757
```

2.3.4) Pesquisas de chaves ausentes

```
B - Search Absent Keys
queries per N = 300000

N=50000 | avg=56,0 ns/contains(absent)
N=100000 | avg=71,2 ns/contains(absent)
N=200000 | avg=79,6 ns/contains(absent)
N=400000 | avg=101,2 ns/contains(absent)
```

2.4) Discussão

Os testes de inserção por blocos mostram que o custo médio de put mantém se baixo e relativamente estável à medida que a tabela cresce, com aumentos pontuais que são compatíveis com períodos de maior custo interno, normalmente associados a redimensionamento ou redistribuição da estrutura.

Nos testes de pesquisa, as razões dobradas ficaram consistentemente abaixo de 2, sugerindo crescimento não linear do tempo médio com o aumento de N, em linha com o comportamento esperado de estruturas de dispersão eficientes em média.

O padrão 80/20 não apresentou melhoria face ao uniforme, pelo contrário, revelou tempos médios superiores nos tamanhos testados.

Por fim, as pesquisas de chaves ausentes exibiram tempos da mesma ordem de grandeza e, para $N = 400\,000$, inferiores aos das pesquisas bem-sucedidas, o que é consistente com a possibilidade de rejeição mais cedo em algumas situações de ausência.

3) Conclusões Geral

Os testes realizados permitem concluir que ambas as estruturas implementadas apresentam um desempenho robusto e consistente com os objetivos do projeto.

No Problema A, a UAlgTree evidenciou um bom nível de balanceamento após inserções massivas, mantendo rácios maxDepth/minDepthestáveis em inserção aleatória (≈1.62–1.83) e ainda mais favoráveis em inserção ordenada (1.3125), indicando resistência clara à degradação típica de árvores de pesquisa simples. Os testes de pesquisa por razão dobrada sugerem crescimento não linear do tempo médio, compatível com comportamento próximo de $O(\log\ N)$. A estratégia de promoção de chaves durante get mostrou benefícios moderados no cenário 80/20, ligados com a ideia de exploração de localidade de acesso, ainda que sem ganhos uniformes em todos os tamanhos testados.

No Problema B, a UAlshTable demonstrou inserções eficientes ao longo do crescimento da estrutura, com custos médios geralmente estáveis e picos pontuais associados a reestruturações internas. As pesquisas apresentaram tempos médios baixos e razões dobradas abaixo de 2, indicando bom desempenho médio típico de estruturas de dispersão. Ao contrário do observado na UAlgTree, o padrão 80/20 não trouxe vantagens claras nesta estrutura, o que é consistente com a natureza de tabelas hash, onde a localidade de acesso não se traduz necessariamente em ganhos de tempo.

Em síntese, a UAlgTree destaca-se pela robustez ao padrão de inserção e pelo potencial de adaptação a cargas com localidade de pesquisa, enquanto a UAlshTable se distingue pela eficiência média de pesquisa e inserção típica de dispersão bem dimensionada. Os resultados obtidos validam as opções de implementação e confirmam o bom funcionamento de ambas as estruturas para cenários de utilização intensiva.